Autómata de un cajero automático

Caso práctico

Una cooperativa de crédito gallega quiere enseñar a sus clientes mayores a usar el cajero, y le pide a **BK Programación** un simulador web. **Carlos** cuenta las pantallas: pedir el PIN, el menú, sacar dinero, ingresar, el comprobante, la despedida... y una tarjeta retenida si se equivoca tres veces. —Son ocho situaciones distintas —dice **María**—, y nueve botones. Sin autómata te vas a perder.

1.- Enunciado

Programa en PHP un **simulador de cajero automático** de una sola cuenta, sin sesiones ni base de datos (todo lo que hay que recordar viaja en campos ocultos):

- Al introducir la tarjeta (cargar la página) se pide el **PIN** (`1234`). Con el PIN correcto se muestra el **menú**; con un PIN incorrecto se avisa de los intentos que quedan y, al **tercer fallo**, la tarjeta queda **retenida**.
- El menú permite **sacar dinero**, **ingresar dinero**, **consultar el saldo** y **salir**.
- Sacar dinero: un importe **múltiplo de 10 €** entre 10 y 300 €, que no supere el **saldo** ni el **límite diario** de 300 € (sumando lo ya sacado hoy). Ingresar: entre 0,01 y 3.000 €, con dos decimales como mucho. Cada pantalla de importe tiene *Confirmar* y *Cancelar* (vuelve al menú).
- Tras cada operación, un **comprobante** con el saldo y los últimos 5 movimientos, con *Otra operación* y *Salir*. *Cancelar* en el PIN o *Salir* muestran la **despedida**.
- La cuenta empieza con **250 €**. Como el saldo viaja en un campo oculto, va **firmado**: si alguien lo cambia en el navegador, el cajero lo detecta y **retiene la tarjeta**.

2.- Análisis

- **Eventos entrantes.** Nueve: la carga de la página ("Introducir la tarjeta") y los ocho botones (`name="accion"`): Aceptar, Retirar, Ingresar, Saldo, Confirmar, Cancelar, Otra operación y Salir.
- **Estados.** Ocho. Cada pantalla que espera algo distinto es un estado: el PIN, el menú, el importe a retirar, el importe a ingresar, el comprobante, la tarjeta retenida y la despedida (más ST0). Retirar e ingresar son estados **distintos** aunque se parezcan: con el mismo botón *Confirmar* hacen cosas diferentes.
- **Variables de estado.** La cuenta (saldo, intentos de PIN, lo retirado hoy y los movimientos) en JSON en un campo oculto, más su **firma**.
- **Predicados.** PL0 (la firma es válida) se comprueba en casi todas las transiciones; los demás son del PIN (PL1, PL2) y de los importes (PL3–PL6). Importante: PL4 (saldo) y PL5 (límite) son **distintos** porque dan avisos distintos.
- **La firma.** `hash_hmac('sha256', estado . '|' . cuenta, SECRETO)` con una clave que solo conoce el servidor. Quien cambia el saldo no puede calcular la firma nueva. Se firma también el **estado**, para que no se pueda saltar, por ejemplo, del PIN al menú.

Debes conocer

En una aplicación real **nunca** se guardaría el saldo en la página: estaría en el servidor (sesión o base de datos, temas 4 y 5). Aquí sirve para practicar la idea de que todo lo que vuelve del navegador puede venir cambiado, y para ver una técnica que sí se usa en la práctica cuando hay que confiar datos al cliente: **firmarlos**.

3.- Diagrama de estado-transición ampliado

Las **cajas discontinuas** representan varios estados a la vez (como STx, pero solo los que nombran): la flecha que sale de ellas vale para todos.

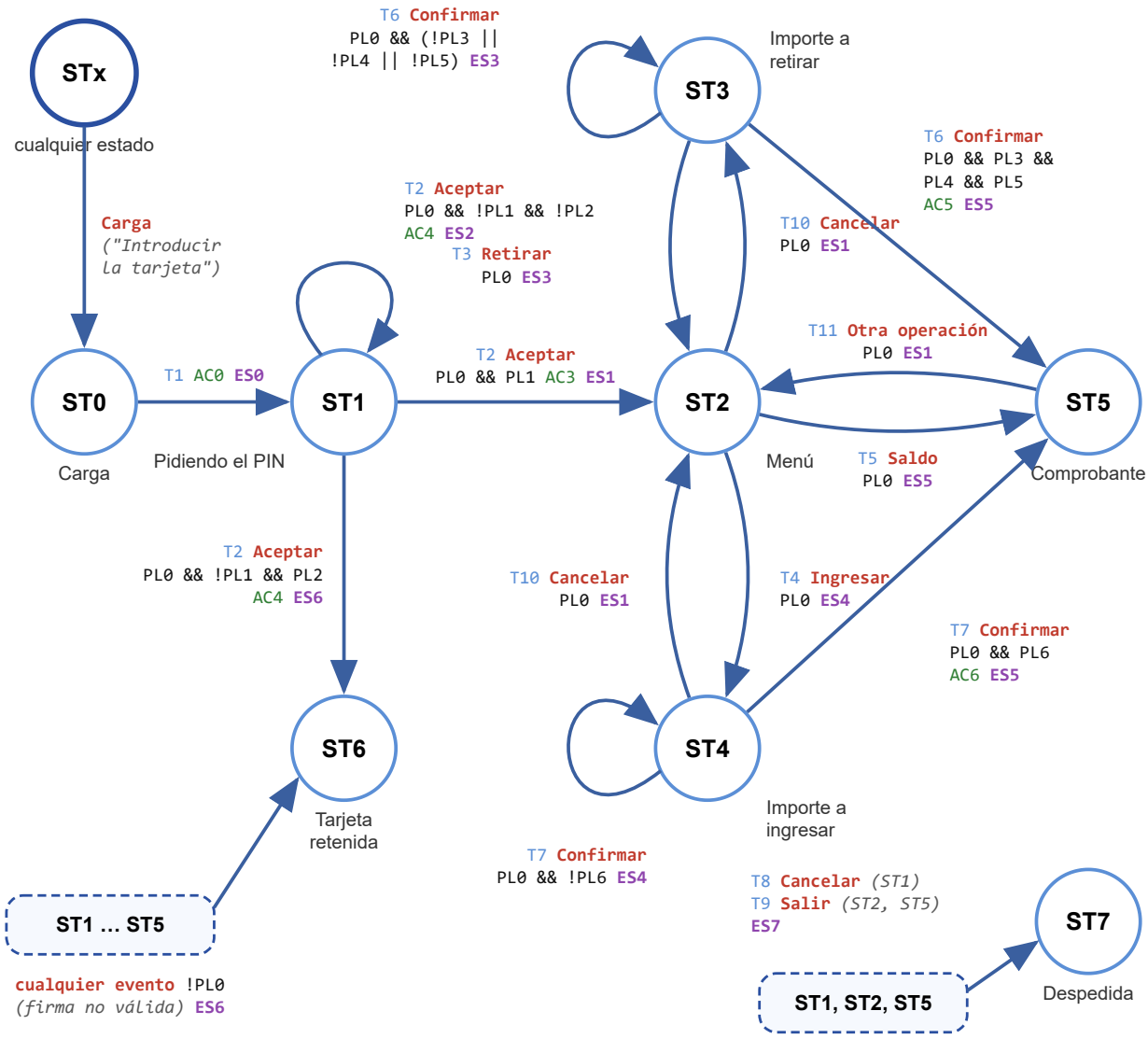


Figura 1. Diagrama del cajero automático

4.- Autómata

4.1.- Tabla de descriptores

Tipo	Abreviatura	Nombre	Significado
Evento entrante	EE0	Carga	página sin parámetros ("Introducir la tarjeta")
Evento entrante	EE1	Aceptar	enviar el PIN
Evento entrante	EE2, EE3, EE4	Retirar, Ingresar, Saldo	opciones del menú
Evento entrante	EE5	Confirmar	confirmar el importe
Evento entrante	EE6	Cancelar	en el PIN: salir; en un importe: volver al menú
Evento entrante	EE7	Otra operación	del comprobante al menú
Evento entrante	EE8	Salir	terminar
Evento saliente	ES0	pantallaPin	"Introduzca su PIN"
Evento saliente	ES1	menu	las cuatro operaciones
Evento saliente	ES2	pinIncorrecto	ES0 con "PIN incorrecto. Le quedan N intentos"
Evento saliente	ES3	importeRetirar	importe a sacar, con el máximo de hoy y un aviso si lo hay
Evento saliente	ES4	importeIngresar	importe a ingresar, con un aviso si lo hay
Evento saliente	ES5	comprobante	operación, saldo y últimos 5 movimientos
Evento saliente	ES6	tarjetaRetenida	con el motivo (3 fallos o datos manipulados)
Evento saliente	ES7	despedida	"Retire su tarjeta"
Estado	ST0	Carga	
Estado	ST1	PidiendoPin	
Estado	ST2	Menu	
Estado	ST3	ImporteRetirar	
Estado	ST4	ImporteIngresar	
Estado	ST5	Comprobante	
Estado	ST6	TarjetaRetenida	
Estado	ST7	Despedida	
Variable de estado	VE0	`$estado`	campo oculto (firmado)
Variable de estado	VE1	`$cuenta`	JSON: saldo (céntimos), intentos, retirado hoy, movimientos
Variable de estado	VE2	`firma`	`hash_hmac` del estado y la cuenta
Variable	VE3, VE4	`$pin`, `$importe`	lo escrito en la pantalla

Tipo	Abreviatura	Nombre	Significado
Predicado	PL0	firma válida	`hash_equals(firma(estado, cuenta), firma recibida)`
Predicado	PL1	PIN correcto	`hash_equals(PIN, $pin)`
Predicado	PL2	último intento	ya llevaba 2 fallos (este es el tercero)
Predicado	PL3	importe a retirar válido	entero, múltiplo de 10, entre 10 y 300
Predicado	PL4	saldo suficiente	importe ≤ saldo
Predicado	PL5	dentro del límite	retirado hoy + importe ≤ 300 €
Predicado	PL6	importe a ingresar válido	de 0,01 a 3.000 €, 2 decimales como mucho
Acción	AC0	cuenta nueva	saldo 250 €, 0 intentos, 0 retirado, sin movimientos
Acción	AC1	recuperar la cuenta	comprobar la firma y decodificar el JSON
Acción	AC2	leer el PIN	
Acción	AC3	`intentos = 0`	
Acción	AC4	`intentos++`	
Acción	AC5	retirar	saldo −= importe; retirado += importe; anotar el movimiento
Acción	AC6	ingresar	saldo += importe; anotar el movimiento
Acción	AC7	leer el importe	
Acción	AC8	firmar y guardar	escribir la cuenta y su firma en campos ocultos (en ST1–ST5)

4.2.- Tabla de eventos-estados ampliada

	Carga	Aceptar	Retirar	Ingresar	Saldo	Confirmar	Cancelar	Otra	Salir
ST0	T1	T1	T1	T1	T1	T1	T1	T1	T1
ST1	T1	T2	---	---	---	---	T8	---	---
ST2	T1	---	T3	T4	T5	---	---	---	T9
ST3	T1	---	---	---	---	T6	T10	---	---
ST4	T1	---	---	---	---	T7	T10	---	---
ST5	T1	---	---	---	---	---	---	T11	T9
ST6	T1	---	---	---	---	---	---	---	---
ST7	T1	---	---	---	---	---	---	---	---

Las celdas `---` son eventos que no pueden darse (no hay ese botón en esa pantalla). Si llegan, se trata como una carga (T1): se vuelve a pedir la tarjeta.

4.3.- Tabla de transiciones

#	AEs antes	PLs	Nuevo ST	AEs después	ESs
T1	---	---	ST1	AC0	ES0 [AC8]
T2	AC1, AC2	`!PL0`	ST6	---	ES6
T2	AC1, AC2	`PL0 && PL1`	ST2	AC3	ES1 [AC8]
T2	AC1, AC2	`PL0 && !PL1 && !PL2`	ST1	AC4	ES2 [AC8]
T2	AC1, AC2	`PL0 && !PL1 && PL2`	ST6	AC4	ES6
T3	AC1	`!PL0` / `PL0`	ST6 / ST3	---	ES6 / ES3 [AC8]
T4	AC1	`!PL0` / `PL0`	ST6 / ST4	---	ES6 / ES4 [AC8]
T5	AC1	`!PL0` / `PL0`	ST6 / ST5	---	ES6 / ES5 [AC8]
T6	AC1, AC7	`!PL0`	ST6	---	ES6
T6	AC1, AC7	`PL0 && !PL3`	ST3	---	ES3 (aviso: múltiplo de 10) [AC8]
T6	AC1, AC7	`PL0 && PL3 && !PL4`	ST3	---	ES3 (aviso: saldo) [AC8]
T6	AC1, AC7	`PL0 && PL3 && PL4 && !PL5`	ST3	---	ES3 (aviso: límite) [AC8]
T6	AC1, AC7	`PL0 && PL3 && PL4 && PL5`	ST5	AC5	ES5 [AC8]
T7	AC1, AC7	`!PL0`	ST6	---	ES6
T7	AC1, AC7	`PL0 && !PL6`	ST4	---	ES4 (aviso) [AC8]
T7	AC1, AC7	`PL0 && PL6`	ST5	AC6	ES5 [AC8]
T8	---	---	ST7	---	ES7
T9	---	---	ST7	---	ES7
T10	AC1	`!PL0` / `PL0`	ST6 / ST2	---	ES6 / ES1 [AC8]
T11	AC1	`!PL0` / `PL0`	ST6 / ST2	---	ES6 / ES1 [AC8]

4.4.- Comprobación de predicados

T2 (el PIN):

PL0	PL1	PL2	`!PL0`	`PL0 && PL1`	`PL0 && !PL1 && !PL2`	`PL0 && !PL1 && PL2`
0	x	x	1	0	0	0
1	1	x	0	1	0	0
1	0	0	0	0	1	0
1	0	1	0	0	0	1

T6 (sacar dinero): los predicados se comprueban **en cascada**. PL4 y PL5 solo tienen sentido con un importe válido (PL3):

PL0	PL3	PL4	PL5	`!PL0`	`!PL3`	`!PL4`	`!PL5`	retirar
0	x	x	x	1	0	0	0	0
1	0	x	x	0	1	0	0	0
1	1	0	x	0	0	1	0	0
1	1	1	0	0	0	0	1	0
1	1	1	1	0	0	0	0	1

Debes conocer

El orden de la cascada decide qué aviso ve el usuario cuando falla más de una cosa: con 200 € de saldo, 150 € ya retirados y una petición de 300 €, fallan PL4 y PL5, pero se avisa del **saldo** porque PL4 va antes. Es una decisión de diseño: escríbela en la tabla para que el código y la documentación digan lo mismo.

Y un detalle de diseño que salió al probar: con un saldo inicial de 500 € y un límite de 300 €, **PL4 nunca podría ser falso** (no se puede pedir más de 300). Un camino que no puede darse es una pista de que algo no cuadra; por eso la cuenta empieza con 250 €.

5.- Del autómata al código

Fragmento de `solucion/index.php`. Cada `elseif` es una fila de la tabla de T6, en el mismo orden:

```
case 6:                                                 // ---- T6 (ST3, Confirmar)
    $cuenta = recuperarCuenta($estado, $_GET);          // AC1
    $escrito = trim((string) ($_GET['importe'] ?? '')); // AC7: leer el importe
    $euros = filter_var($escrito, FILTER_VALIDATE_INT);
    $importe = $euros === false ? 0 : $euros * 100;
    if ($cuenta === null) {                             // !PL0
        $estado = ST6;
        es6($retenida);
    } elseif ($euros === false || $euros < 10 || $euros > LIMITE_DIARIO / 100 || $euros % 10
!== 0) {   // PL0 && !PL3
        $estado = ST3;
        es3($cuenta, $escrito, 'El importe debe ser un múltiplo de 10 € entre 10 y 300 €.');
    } elseif ($importe > $cuenta['saldo']) {            // PL0 && PL3 && !PL4
        $estado = ST3;
        es3($cuenta, $escrito, 'No tiene saldo suficiente.');
    } elseif ($cuenta['retirado'] + $importe > LIMITE_DIARIO) {   // PL0 && PL3 && PL4 && !PL5
        $estado = ST3;
        es3($cuenta, $escrito, 'Supera el límite diario: hoy puede sacar ' .
euros(LIMITE_DIARIO - $cuenta['retirado']) . '.');
    } else {                                            // PL0 && PL3 && PL4 && PL5
        $estado = ST5;
        $cuenta['saldo'] -= $importe;                   // AC5: retirar
        $cuenta['retirado'] += $importe;
        $cuenta = anotar($cuenta, 'Reintegro', -$importe);
        es5($cuenta, 'Retire su dinero: ' . euros($importe));
    }
    break;
```

La firma (AC1 y AC8):

```
function firma(int $estado, string $cuentaJson): string
{
    return hash_hmac('sha256', $estado . '|' . $cuentaJson, SECRETO);
}

function recuperarCuenta(int $estado, array $datos): ?array      // AC1: null si PL0 es falso
{
    $json = (string) ($datos['cuenta'] ?? '');
    $recibida = (string) ($datos['firma'] ?? '');
    if (!hash_equals(firma($estado, $json), $recibida)) {
        return null;
    }
    $cuenta = json_decode($json, true);
    return is_array($cuenta) ? $cuenta : null;
}
```

6.- Pruebas

Se hicieron 31 peticiones reenviando cada vez el formulario completo, como el navegador. Algunos caminos:

Prueba	Camino	Resultado comprobado
PIN 0000, 9999 y 1111	T2: `!PL1 && !PL2` dos veces y `!PL1 && PL2`	"Le quedan 2 intentos", "Le queda 1 intento", tarjeta retenida
PIN 1234 y Saldo	T2 `PL1`, T5	menú; comprobante con 250,00 €
Retirar 25 €	T6 `!PL3`	"múltiplo de 10 € entre 10 y 300 €"
Retirar 300 €	T6 `!PL4`	"No tiene saldo suficiente"
Retirar 200 €	T6 retirar	saldo 50,00 € y movimiento "Reintegro"
Ingresar "abc" y "500,5"	T7 `!PL6`, T7 `PL6`	aviso; ingreso de 500,50 € (saldo 550,50 €)
Retirar 150 €	T6 `!PL5`	"Supera el límite diario: hoy puede sacar 100,00 €"
Cancelar en el importe	T10	menú
saldo cambiado a 999999,99 € en el campo oculto	`!PL0`	tarjeta retenida "página manipulada"
estado 2 en la URL sin firma válida	T5 `!PL0`	tarjeta retenida
Cancelar en el PIN; Salir del menú	T8, T9	"Retire su tarjeta"
Aceptar en ST6	`---` → T1	se vuelve a pedir el PIN
importe `<script>`	T7 `!PL6`	escapado

7.- Para practicar

- Añade la opción **Cambiar el PIN** (pedir el actual y dos veces el nuevo). ¿Cuántos estados nuevos? ¿Qué predicados? Rellena su fila y su columna en la tabla de eventos-estados.
- Haz que tras 20 segundos sin hacer nada se expulse la tarjeta. ¿Es un evento entrante? ¿Quién lo genera (pista: JavaScript con `setTimeout` enviando el formulario)?
